
Rapport de DAAR

WebApp – Moteur de Recherche d'une Bibliothèque

MENGXIAO LI(21304592)
XUE YANG(28705179)
LIU YANG(21512070)

UE DAAR
MASTER INFORMATIQUE - STL

Tuteur(s) université :
BINH-MINH BUI-XUAN
MASSINISSA TIGHILT

Novembre 2025

Table des matières

1	Introduction	1
2	Indexation et algorithmes du moteur de recherche	1
2.1	Objectifs de conception et architecture générale	1
2.2	Structure de la base de données	2
2.3	Modèle TF-IDF et construction de l'index inversé	2
2.3.1	Définition du problème et structures de données utilisées	2
2.3.2	Définitions mathématiques de TF, DF, IDF et TF-IDF	3
2.3.3	Avantages, limites et améliorations	3
2.3.4	Utilisation du modèle dans le projet	4
2.4	Vectorisation des documents et similarité cosinus	4
2.4.1	Définition du problème	5
2.4.2	Définition mathématique de la similarité cosinus	5
2.4.3	Contexte théorique et justification du choix	5
2.4.4	Utilisation de la similarité cosinus dans le projet	6
2.5	Mesures de centralité	7
2.5.1	Définition du problème	7
2.5.2	Algorithmes de centralité	7
2.5.3	Utilisation des centralités dans le projet	8
3	Tests, validation et performance du moteur	9
3.1	Méthodologie des tests et testbeds	9
3.2	Tests du modèle TF-IDF et de l'index inversé	9
3.2.1	Distribution des longueurs de documents	9
3.2.2	Distribution des fréquences documentaires (DF)	10
3.2.3	Distribution de la sparsité des vecteurs TF-IDF	10
3.3	Tests de similarité cosinus	11
3.3.1	Distribution des similarités cosinus	11
3.3.2	Choix du seuil (threshold) pour la construction du graphe	11
3.4	Tests et analyse des mesures de centralité	12
3.4.1	Distribution de la centralité de degré (Popularity)	12
3.4.2	Distribution de la centralité de proximité (Closeness)	13
3.4.3	Distribution de la centralité d'intermédiarité (Betweenness)	13
3.4.4	Distribution du PageRank	13
4	Conclusion	13

1 Introduction

Ce projet met en œuvre un moteur de recherche textuelle conçu pour un corpus anglophone de plus de 1600 ouvrages issus de Project Gutenberg. L'application, développée avec Django, fournit une interface Web légère qui permet de tester facilement plusieurs modes de recherche : recherche par mots-clés fondée sur le modèle TF-IDF, recherche par expressions régulières reposant sur un moteur DFA implémenté pour le projet, filtrage direct par titre ou par auteur, ainsi qu'un module de recommandation basé sur la similarité entre documents. Toutes ces fonctionnalités sont exposées à travers une API backend unifiée et présentées dans l'interface Web.

Bien que le système comporte une interface complète et des fonctionnalités d'interaction, le présent rapport se concentre principalement sur les algorithmes qui constituent le cœur du moteur de recherche. Les sections suivantes détaillent :

- **Le modèle TF-IDF et la construction de l'index inversé** : statistiques TF et DF, définition mathématique et implantation ;
- **La vectorisation des documents et la similarité cosinus** : représentation vectorielle, calcul de la similarité et choix de seuils ;
- **Le graphe documentaire et les mesures de centralité** : Degree, Closeness, Betweenness et PageRank, leurs interprétations et usages ;
- **Les tests et analyses de performance** : distributions des termes, sparsité des vecteurs TF-IDF, distribution des similarités et des centralités.

En examinant la définition, les fondements théoriques, les choix d'implémentation et les résultats expérimentaux de ces algorithmes, ce rapport vise à présenter de manière claire les mécanismes sous-jacents du moteur de recherche ainsi que leur comportement effectif sur un corpus réel.

Accès au Book Engine

Le moteur de recherche est disponible sous deux formes :

- **Dépôt GitHub** : https://github.com/yangl8/DAAR_Book_Engine

Le dépôt contient l'intégralité du code source, le script de construction de la base d'indexation ainsi que les instructions d'exécution.

- **Version en ligne** : <https://daar-book-engine.onrender.com>

Hébergement gratuit ; le premier chargement peut prendre environ 2 minutes (démarrage du serveur). Pour de meilleures performances, nous recommandons l'exécution locale.

2 Indexation et algorithmes du moteur de recherche

Cette section décrit le pipeline d'indexation qui permet de transformer les documents bruts en structures exploitables pour la recherche et le classement. Elle présente successivement : (i) les objectifs de conception du système, (ii) l'organisation de la base de données, (iii) le modèle TF-IDF et l'index inversé, (iv) la vectorisation des documents et la similarité cosinus, et (v) les mesures de centralité dans le graphe documentaire.

L'ensemble du pipeline repose sur `SQLite` pour le stockage, et sur l'ORM Django pour l'accès unifié aux données. La base peut être reconstruite automatiquement grâce à un script dédié, assurant la reproductibilité complète du système.

2.1 Objectifs de conception et architecture générale

Cette section présente les algorithmes qui constituent le cœur du moteur de recherche : construction de l'index inversé, pondération TF-IDF, mesure de similarité cosinus et calcul

des centralités dans le graphe documentaire. Le système de stockage sert uniquement de support pour ces traitements, et les objectifs de conception sont centrés sur les besoins de ces algorithmes :

1. Support d'un index inversé à grande échelle

La base doit permettre d'enregistrer les termes, leurs fréquences (TF, DF) et les pondérations TF-IDF, constituant la structure fondamentale de la recherche en texte intégral.

2. Support du modèle vectoriel pour la comparaison de documents

Chaque document doit pouvoir être représenté sous forme de vecteur TF-IDF, afin de calculer efficacement la similarité cosinus et de construire le graphe documentaire G_d .

3. Support des mesures de centralité dans le graphe

Les scores de centralité (degree, closeness, betweenness, PageRank) doivent pouvoir être stockés et exploités pour améliorer le classement et la recommandation de documents.

Sur cette base, le pipeline général d'indexation suivi dans le projet est le suivant :

Prétraitement du corpus $\rightarrow$ *Construction de l'index (TF, DF, TF-IDF)* $\rightarrow$ *Vectorisation des documents* $\rightarrow$ *Calcul de la similarité cosinus* $\rightarrow$ *Construction du graphe documentaire* $\rightarrow$ *Mesures de centralité*

Cette architecture modulaire permet d'intégrer proprement chaque algorithme et d'assurer la cohérence du moteur de recherche dans son ensemble.

2.2 Structure de la base de données

La base de données **SQLite** comporte cinq tables principales :

- **books** : métadonnées des ouvrages (titre, auteurs, longueur, chemin local) ;
- **terms** : vocabulaire normalisé et fréquences documentaires (DF) ;
- **postings** : index inversé, stockant les couples (term, document) avec TF et TF-IDF ;
- **document_graph** : arêtes pondérées du graphe de similarité (cosinus) ;
- **document_scores** : mesures de centralité (degree, closeness, betweenness, PageRank).

Cette structure minimaliste permet de stocker efficacement toutes les informations nécessaires à l'indexation, au calcul de similarité et à l'analyse du graphe documentaire.

2.3 Modèle TF-IDF et construction de l'index inversé

Après le prétraitement du corpus, le projet utilise le modèle de pondération TF-IDF afin de représenter chaque document sous forme vectorielle, puis génère un index inversé à partir de ces pondérations. Cet index constitue non seulement la structure centrale permettant de répondre efficacement aux requêtes des utilisateurs, mais aussi la base des étapes suivantes : vectorisation des documents, calcul de similarité et construction du graphe documentaire.

2.3.1 Définition du problème et structures de données utilisées

Dans un corpus composé de N documents $\{d_1, d_2, \dots, d_N\}$, l'objectif est de construire pour chaque document une représentation vectorielle capable de refléter l'importance relative de ses termes. Cette représentation doit permettre de comparer les documents entre eux, d'effectuer des calculs de similarité et de répondre efficacement aux requêtes de recherche.

Pour rendre ces opérations possibles, le système repose sur deux structures de données essentielles :

- **La table terms** : elle enregistre l'ensemble des termes normalisés du corpus ainsi que leur fréquence documentaire (DF).

- **La table postings** : elle contient, pour chaque paire (terme, document), la fréquence d'apparition (TF) ainsi que le poids TF-IDF correspondant.

Ces deux structures forment la base de l'index inversé, permettant de retrouver rapidement les documents contenant un terme donné, tout en servant de fondation aux calculs de similarité et à la construction ultérieure du graphe documentaire.

2.3.2 Définitions mathématiques de TF, DF, IDF et TF-IDF

La pondération TF-IDF repose sur quatre quantités fondamentales permettant d'estimer l'importance d'un terme dans un document et dans l'ensemble du corpus.

Term Frequency (TF) La fréquence d'apparition d'un terme t dans un document d est définie par :

$$\text{TF}(t, d) = \text{nombre d'occurrences de } t \text{ dans } d.$$

Document Frequency (DF) La fréquence documentaire d'un terme correspond au nombre de documents dans lesquels il apparaît :

$$\text{DF}(t) = |\{d \in D \mid t \in d\}|.$$

Inverse Document Frequency (IDF) L'IDF mesure la rareté d'un terme dans le corpus. Nous utilisons la forme logarithmique classique :

$$\text{IDF}(t) = \log\left(\frac{N}{\text{DF}(t)}\right),$$

où N désigne le nombre total de documents du corpus.

Poids TF-IDF Le poids final associé à un terme t dans un document d est donné par :

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t).$$

Ce poids est ensuite stocké dans la table **postings** et sert de base à la vectorisation des documents ainsi qu'au calcul de similarité.

2.3.3 Avantages, limites et améliorations

Le modèle TF-IDF constitue l'un des schémas de pondération les plus utilisés en recherche d'information, en particulier pour les corpus textuels volumineux et clairsemés tels que les ouvrages du Project Gutenberg. Dans ce projet, nous analysons ses avantages fondamentaux, ses limites, ainsi que les améliorations intégrées dans notre pipeline d'indexation.

Avantages principaux :

- **Forte capacité de discrimination** : les termes fréquents dans un document mais rares dans le corpus obtiennent un poids élevé, ce qui améliore la séparation thématique entre les ouvrages.
- **Coût computationnel réduit** : les calculs reposent sur des opérations simples, compatibles avec un traitement à grande échelle sans ressources matérielles particulières.
- **Interprétabilité élevée** : les composantes TF et IDF possèdent une signification statistique claire, facilitant l'analyse et le contrôle de la qualité de l'index.
- **Compatibilité naturelle avec le modèle vectoriel** : les vecteurs TF-IDF s'intègrent directement au calcul de similarité cosinus et constituent la base de la construction du graphe documentaire.

Limites et critiques :

- **Absence de prise en compte de la sémantique** : les mots synonymes (par ex. *car* / *automobile*) sont traités comme des termes complètement distincts.
- **Amplification des termes rares** : les erreurs d'OCR ou les fautes d'orthographe peuvent recevoir un IDF anormalement élevé.
- **Modèle sans contexte** : TF-IDF ignore l'ordre des mots et la structure syntaxique, limitant la capture d'informations sémantiques plus riches.
- **Biais lié à la longueur des documents** : les textes longs produisent naturellement des valeurs TF plus élevées, ce qui peut influencer les pondérations sans normalisation supplémentaire.

Améliorations intégrées dans ce projet :

- **Filtrage des stopwords** pour réduire l'influence des mots outils.
- **Stemming (Porter)**, permettant d'unifier les variantes morphologiques (ex. *run*, *runs*, *running*).
- **Filtrage Top-K par document** afin de limiter les termes très peu fréquents dans les textes longs et de réduire le bruit lexical.
- **Filtrage global basé sur DF** : suppression des termes apparaissant dans plus de 95% des documents ainsi que ceux présents dans deux documents ou moins.
- **Recalcul systématique du DF** et suppression des postings orphelins, garantissant la cohérence de l'index final.
- **Filtrage Top-K basé sur TF-IDF** : après le calcul des poids TF-IDF, seuls les 2500 termes les plus informatifs sont conservés pour chaque document, afin d'obtenir des vecteurs plus compacts et de réduire les coûts de similarité et de calcul du graphe.

2.3.4 Utilisation du modèle dans le projet

Dans notre pipeline, TF-IDF n'est pas uniquement un modèle théorique, mais un élément central de la construction de l'index inversé. Après le prétraitement et la tokénisation, les fréquences TF sont intégrées dans la table `postings`, tandis que les fréquences documentaires DF sont stockées dans la table `terms`. Les poids TF-IDF calculés à partir de ces valeurs sont inscrits dans le champ `tfidf` de la table `postings`.

Les vecteurs TF-IDF ainsi obtenus constituent la représentation vectorielle finale de chaque document. Ils servent de base :

- au calcul de la similarité cosinus ;
- à la construction du graphe documentaire G_d ;
- au calcul des mesures de centralité (PageRank, Closeness, Betweenness, Degree).

Ainsi, TF-IDF joue un rôle structurant dans l'ensemble du moteur de recherche, en reliant la phase d'indexation lexicale aux étapes de comparaison, de modélisation du graphe et de recommandation.

2.4 Vectorisation des documents et similarité cosinus

Après le calcul des pondérations TF-IDF, le système récupère pour chaque document l'ensemble des couples (`term_id`, `tfidf`) stockés dans la table `postings`, puis les organise sous forme de vecteurs creux. Ces vecteurs TF-IDF constituent une représentation efficace du contenu des documents, adaptée aux données textuelles de grande dimension.

Une fois ces vecteurs construits, il est nécessaire de disposer d'une mesure capable d'évaluer la proximité thématique entre deux documents. La similarité cosinus (*cosine similarity*) est la mesure la plus répandue dans le modèle vectoriel, en particulier pour les vecteurs TF-IDF creux. Elle permet d'estimer la proximité entre documents en comparant l'angle entre leurs vecteurs,

indépendamment de leur longueur, ce qui en fait un élément essentiel pour la construction du graphe documentaire et pour le module de recommandation.

2.4.1 Définition du problème

Dans le modèle vectoriel, chaque document d est représenté par un vecteur TF-IDF $\vec{d}$, dont chaque dimension correspond au poids d'un terme dans le document. Pour comparer deux documents, nous souhaitons disposer d'une mesure qui vérifie les propriétés suivantes :

- plus les documents partagent de termes importants, plus la similarité doit être élevée ;
- plus leurs thèmes diffèrent, plus la similarité doit être faible ;
- la longueur des documents ne doit pas biaiser la comparaison ;
- la mesure doit rester stable dans des espaces de grande dimension et fortement clairsemés.

La comparaison des documents à l'aide de l'angle entre leurs vecteurs répond naturellement à ces exigences, ce qui justifie l'utilisation de la similarité cosinus dans notre projet.

2.4.2 Définition mathématique de la similarité cosinus

Pour deux vecteurs documentaires $\vec{d}_1$ et $\vec{d}_2$, la similarité cosinus est définie par :

$$\text{sim}(\vec{d}_1, \vec{d}_2) = \frac{\vec{d}_1 \cdot \vec{d}_2}{\|\vec{d}_1\| \|\vec{d}_2\|}.$$

Cette quantité se lit comme suit :

- le numérateur $\vec{d}_1 \cdot \vec{d}_2$ correspond au produit scalaire des deux vecteurs, mesurant le degré de recouvrement lexical ;
- le dénominateur normalise la longueur des vecteurs, empêchant les documents longs d'obtenir artificiellement une similarité plus élevée ;
- pour des vecteurs TF-IDF (non négatifs), la similarité appartient à l'intervalle $[0, 1]$.

Cette mesure permet donc d'évaluer la proximité thématique entre documents sans être influencée par leur taille, ce qui la rend particulièrement adaptée aux corpus littéraires volumineux et hétérogènes utilisés dans ce projet.

2.4.3 Contexte théorique et justification du choix

Dans ce projet, nous avons envisagé deux mesures de similarité possibles pour évaluer la proximité entre documents : la similarité de Jaccard, fondée sur le chevauchement des ensembles de termes, et la similarité cosinus, issue du modèle vectoriel. Bien que ces deux approches puissent être utilisées pour comparer des textes, l'analyse théorique et expérimentale nous a conduits à privilégier la similarité cosinus pour les raisons suivantes.

a) La similarité cosinus exploite pleinement les pondérations TF-IDF, contrairement à Jaccard. La similarité de Jaccard repose uniquement sur la présence ou l'absence d'un terme dans un document, sans tenir compte ni de sa fréquence (TF), ni de son importance statistique (IDF). Ainsi, un terme fréquent et informatif et un terme rare mais essentiel sont traités de la même manière. À l'inverse, la similarité cosinus s'appuie directement sur les vecteurs pondérés TF-IDF, ce qui permet de refléter la contribution réelle de chaque terme à la représentation du document.

b) La similarité cosinus neutralise naturellement l'effet de la longueur des documents, ce que Jaccard ne permet pas. Les textes du Project Gutenberg présentent des longueurs très hétérogènes, allant de quelques milliers à plusieurs centaines de milliers de mots.

La mesure de Jaccard est sensible à cette variation, car la taille de l'union des termes augmente fortement pour les documents longs, ce qui entraîne une similarité artificiellement faible.

La similarité cosinus, en normalisant les vecteurs par leurs normes, élimine cet effet et compare les documents uniquement selon la direction de leurs vecteurs TF-IDF, ce qui est plus pertinent dans ce contexte.

c) Pour des vecteurs TF-IDF de grande dimension et fortement clairsemés, la similarité cosinus est plus stable et mieux discriminante. Les vecteurs TF-IDF du corpus sont de dimension élevée et très clairsemés : la plupart des termes apparaissent uniquement dans une minorité de documents. Dans ces conditions, la similarité de Jaccard aboutit souvent à des valeurs proches de zéro (faible intersection, union très large), ce qui rend les documents difficiles à distinguer et mène à un graphe documentaire fragmenté.

La similarité cosinus, fondée sur le produit scalaire des termes partagés, produit au contraire une distribution de similarités plus riche et plus stable, ce qui facilite la construction d'un graphe documentaire exploitable.

d) La similarité cosinus fournit des poids continus adaptés au graphe documentaire G_d . Le graphe documentaire nécessite des arêtes pondérées reflétant la proximité réelle entre documents. La similarité de Jaccard, souvent quasi nulle dans les corpus clairsemés, offre des poids très discrets et peu informatifs.

La similarité cosinus génère des poids continus et bien différenciés, ce qui améliore la qualité des arêtes du graphe et permet d'obtenir des résultats plus fiables lors du calcul des mesures de centralité (PageRank, Closeness, Betweenness).

2.4.4 Utilisation de la similarité cosinus dans le projet

Dans ce projet, la similarité cosinus joue un rôle central dans le calcul de la proximité entre documents et dans la construction du graphe documentaire G_d . Son utilisation s'articule principalement autour de deux étapes.

a) Calcul de la similarité entre documents à partir des vecteurs TF-IDF Après le calcul des pondérations TF-IDF, le système extrait pour chaque document l'ensemble des couples (`term_id`, `tfidf`) depuis la table `postings` et construit un vecteur creux :

```
vector = {p.term_id: p.tfidf for p in postings}
```

La similarité cosinus entre deux documents d_i et d_j est ensuite obtenue en :

- calculant le produit scalaire uniquement sur les termes communs ;
- normalisant chaque vecteur par sa norme afin d'éliminer l'effet de la longueur des documents ;
- produisant une valeur de similarité dans l'intervalle $[0, 1]$.

Cette méthode permet une comparaison efficace et fidèle aux pondérations TF-IDF.

b) Construction du graphe documentaire G_d Les similarités cosinus ainsi obtenues servent directement à construire un graphe documentaire pondéré. Le système compare chaque paire de documents :

```
sim = cosine(vectors[idA], vectors[idB])
if sim >= threshold:
    DocumentGraph.objects.create(doc1_id=idA, doc2_id=idB, similarity=sim)
```


Lorsque la similarité dépasse un seuil fixé (par exemple 0.05), une arête est ajoutée dans la table `document_graph`.

Le graphe G_d résultant est un graphe non orienté et pondéré, suffisamment clairsemé mais bien connecté, qui sert de base :

- au calcul des mesures de centralité (PageRank, Closeness, Betweenness) ;
- au module de recommandation, qui exploite les voisins les plus proches des documents seed.

2.5 Mesures de centralité

Dans le graphe documentaire $G_d = (V, E)$, chaque nœud représente un document et chaque arête pondérée exprime la similarité cosinus entre deux documents. Afin d'évaluer l'importance structurelle des documents dans ce graphe, nous utilisons quatre mesures de centralité classiques : la centralité de degré, la centralité de proximité, la centralité d'intermédierité et le PageRank. Ces mesures caractérisent respectivement la connectivité locale, la position globale, le rôle de « pont » entre communautés, et l'influence structurelle d'un document.

2.5.1 Définition du problème

Dans le graphe documentaire G_d , la similarité cosinus permet de mesurer les relations locales entre documents, mais elle ne suffit pas à décrire leur importance structurelle dans l'ensemble du corpus. Deux documents peuvent avoir une similarité élevée tout en occupant des positions très différentes dans le graphe. Pour obtenir une vision plus globale de l'organisation thématique du corpus, il est nécessaire d'analyser la place qu'occupe chaque document dans la structure du graphe.

Les mesures de centralité apportent cette information complémentaire en évaluant :

- l'influence globale d'un document dans le corpus ;
- son rôle potentiel de « pont » entre plusieurs communautés thématiques ;
- sa capacité à représenter un thème central ou, au contraire, à se situer en périphérie du graphe.

Ainsi, l'analyse de centralité enrichit la simple mesure de similarité en fournissant une perspective structurelle indispensable au classement des documents et à la qualité des recommandations.

2.5.2 Algorithmes de centralité

Centralité de degré : La centralité de degré mesure le nombre de voisins d'un nœud, c'est-à-dire le nombre de documents présentant une forte similarité avec lui. Elle reflète l'importance *locale* d'un document dans le graphe.

$$C_D(v) = \frac{\deg(v)}{|V| - 1}$$

où $\deg(v)$ désigne le degré du nœud v .

Centralité de proximité(Closeness) : La centralité de proximité évalue la distance moyenne d'un nœud à l'ensemble des autres nœuds du graphe. Un document ayant une proximité élevée est situé près du « centre » structurel du corpus.

$$C_C(v) = \frac{1}{\sum_{u \in V} d(v, u)}$$

où $d(v, u)$ représente la distance du plus court chemin entre v et u .

Centralité d'intermédiarité(Betweenness) : La centralité d'intermédiarité mesure la fréquence à laquelle un nœud intervient sur les plus courts chemins reliant des paires de nœuds. Elle permet d'identifier les documents jouant un rôle de *pont* entre différentes communautés thématiques.

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

où σ_{st} désigne le nombre de plus courts chemins reliant s à t , et $\sigma_{st}(v)$ le nombre de ces chemins passant par v .

PageRank : Le PageRank repose sur l'idée que l'importance d'un nœud est renforcée lorsqu'il est connecté à des nœuds eux-mêmes importants. Il s'inspire d'un modèle de marche aléatoire sur le graphe.

Le PageRank classique est défini par :

$$PR(v) = \alpha \sum_{u \in N(v)} \frac{PR(u)}{\deg(u)} + (1 - \alpha) \frac{1}{|V|}$$

Pour un graphe pondéré — comme notre graphe où les poids correspondent aux similarités cosinus — la formule devient :

$$PR(v) = \alpha \sum_{u \in N(v)} \frac{w_{uv}}{\sum_{t \in N(u)} w_{ut}} PR(u) + (1 - \alpha) \frac{1}{|V|}$$

où w_{uv} est le poids (similarité cosinus) de l'arête entre u et v .

2.5.3 Utilisation des centralités dans le projet

Après le calcul des quatre mesures de centralité (Degree, Closeness, Betweenness et PageRank) sur le graphe documentaire G_d , une phase supplémentaire est nécessaire afin de rendre ces valeurs comparables et exploitables dans les composants du moteur de recherche et du système de recommandation. Cette phase comprend deux étapes : la normalisation et la construction d'un score global.

a) Normalisation des centralités Les différentes mesures de centralité présentent des échelles numériques très variées : le PageRank présente en général une amplitude faible, tandis que la Betweenness peut atteindre des valeurs élevées, et les centralités de degré et de proximité occupent encore d'autres intervalles. Pour assurer une pondération cohérente, nous appliquons une normalisation min-max :

$$C^{\text{norm}}(v) = \frac{C(v) - \min C}{\max C - \min C}.$$

Cette normalisation projette toutes les valeurs dans l'intervalle $[0, 1]$ et garantit :

- la comparabilité directe entre les quatre mesures ;
- la stabilité numérique lors de la combinaison ;
- une pondération équilibrée dans le score final.

b) Construction du score global (*total score*) Une fois les valeurs normalisées, nous définissons pour chaque document un score global reflétant son importance structurelle dans le graphe. Ce score correspond à une combinaison linéaire pondérée :

$$\text{total}(v) = 0.1 \cdot C_D(v) + 0.2 \cdot C_C(v) + 0.2 \cdot C_B(v) + 0.5 \cdot PR(v).$$

La pondération attribuée à chaque centralité répond à des considérations structurelles :

- **PageRank (0.5)** : indicateur de l'influence globale, le plus robuste et donc le plus pondéré ;
- **Closeness (0.2)** : mesure la position du document par rapport à l'ensemble du corpus ;
- **Betweenness (0.2)** : valorise les documents jouant un rôle d'intermédiaire entre différentes communautés thématiques ;
- **Degree (0.1)** : reflète la connectivité locale du document.

c) Stockage et utilisation Les valeurs de chaque centralité ainsi que le score global sont stockées dans la table `document_scores`. Elles sont utilisées dans :

1. **le classement des résultats de recherche**, où le score global complète la similarité TF-IDF pour obtenir un tri plus pertinent ;
2. **le système de recommandation**, où les documents les plus centraux sont privilégiés parmi les voisins des documents *seed*.

3 Tests, validation et performance du moteur

3.1 Méthodologie des tests et testbeds

L'ensemble des tests présentés dans ce rapport s'appuie sur un corpus réel provenant du catalogue officiel de Project Gutenberg. À partir du fichier `pg_catalog.csv`, nous sélectionnons exclusivement les ouvrages en anglais en nous basant sur le champ `Language`. Pour chaque identifiant, plusieurs versions HTML sont testées afin de sélectionner un fichier disposant d'un préambule complet (informations bibliographiques et licence). Le texte brut est ensuite extrait en supprimant les balises techniques tout en conservant la structure des paragraphes.

Seule la portion comprise entre *START OF ...* et *END OF ...* est retenue pour l'analyse. Les ouvrages dont le corps textuel contient au moins 10 000 mots sont conservés. Ce processus permet d'obtenir un testbed constitué de **1664 ouvrages** accompagnés de leurs métadonnées (identifiant Gutenberg, titre, auteurs, URL source, longueur du texte).

3.2 Tests du modèle TF-IDF et de l'index inversé

Cette section analyse les propriétés statistiques essentielles du corpus après prétraitement et construction de l'index inversé. Les tests portent principalement sur : (1) la distribution des longueurs de documents, (2) la distribution des fréquences documentaires (DF), et (3) la sparsité des vecteurs TF-IDF. Ces mesures permettent de vérifier la cohérence du corpus, la stabilité du pipeline de construction et la pertinence des choix algorithmiques.

3.2.1 Distribution des longueurs de documents

Après nettoyage, tokenisation, suppression des stopwords et racinisation (Porter Stemmer), chaque document est représenté par un nombre effectif de tokens. La répartition de ces longueurs constitue un indicateur fondamental du testbed.

Comme le montre la Fig. 1, les longueurs présentent une distribution fortement asymétrique : la majorité des ouvrages se situent entre 10 000 et 100 000 tokens, tandis qu'un petit nombre

dépasse plusieurs centaines de milliers de tokens. Cette hétérogénéité confirme la variété du corpus et garantit que le modèle TF-IDF est évalué sur des documents de tailles très différentes.

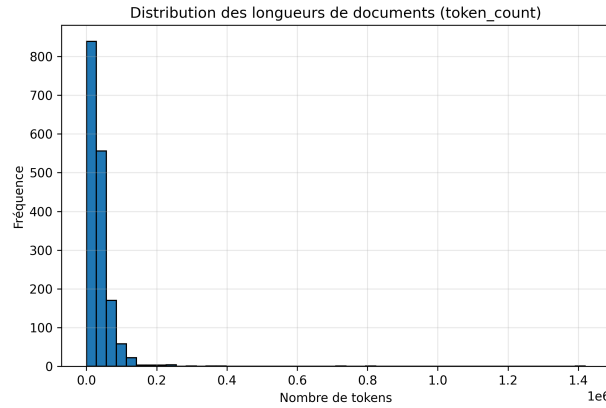


FIGURE 1 – Distribution des longueurs de documents (token count)

3.2.2 Distribution des fréquences documentaires (DF)

La fréquence documentaire (DF) mesure le nombre de documents dans lesquels un terme apparaît. Elle joue un rôle central dans le calcul de l’IDF et donc dans l’importance attribuée à chaque terme.

La Fig. 2 révèle une distribution typiquement en loi de Zipf : une très grande majorité de termes possèdent un DF faible (entre 1 et 10), tandis qu’un petit nombre apparaît dans plusieurs centaines de documents. Cette structure confirme la cohérence statistique du corpus et justifie l’application de filtres de haute et basse fréquence ($DF \geq 95\%$ et $DF \leq 2$).

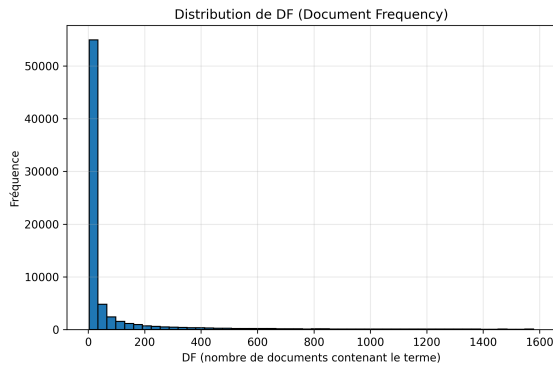


FIGURE 2 – Distribution des fréquences documentaires (DF)

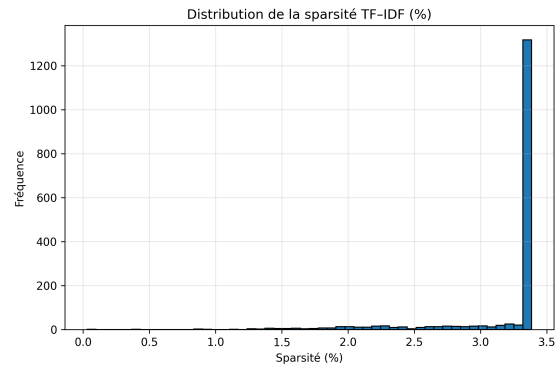


FIGURE 3 – Distribution de la sparsité des vecteurs TF-IDF

3.2.3 Distribution de la sparsité des vecteurs TF-IDF

Après construction du modèle TF-IDF, chaque document est représenté par un vecteur creux obtenu en conservant uniquement ses top-K termes ($K = 2000$). Afin d’évaluer la structure de ces vecteurs, nous calculons la sparsité :

$$\text{sparsité} = \frac{\text{nombre de coefficients non nuls}}{\text{taille du vocabulaire}} \times 100\%.$$

La Fig. 3 montre que la sparsité des documents est fortement concentrée entre 2% et 3.5%. Cette très forte parcimonie confirme que le modèle TF-IDF produit bien des vecteurs haute dimension mais extrêmement creux, ce qui rend la similarité cosinus particulièrement adaptée.

3.3 Tests de similarité cosinus

La similarité cosinus constitue la mesure centrale permettant d'évaluer la proximité sémantique entre documents et de construire le graphe documentaire G_d . Dans cette section, nous analysons (1) la distribution empirique des similarités, et (2) la justification du seuil utilisé pour filtrer les arêtes du graphe.

3.3.1 Distribution des similarités cosinus

Nous avons calculé la similarité cosinus pour toutes les paires de documents ayant une similarité ≥ 0.05 , puis représenté leur distribution sous forme d'histogramme (voir Fig. 4).

Les résultats montrent une distribution fortement asymétrique :

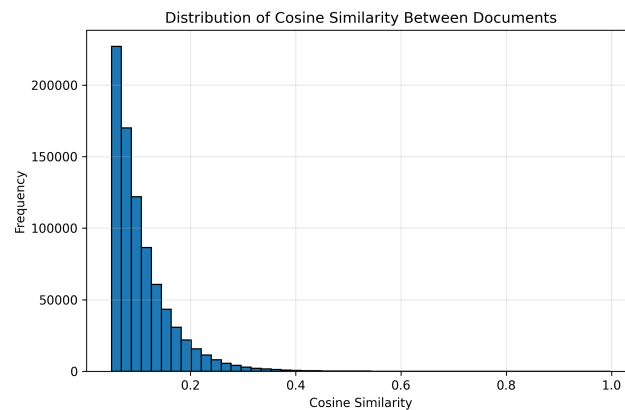


FIGURE 4 – Distribution des similarités cosinus entre documents

- la grande majorité des paires de documents présentent une similarité comprise entre 0.05 et 0.15 ;
- la fréquence diminue rapidement lorsque la similarité augmente ;
- très peu de paires dépassent 0.30 ;
- les similarités supérieures à 0.5 sont extrêmement rares et correspondent généralement à des ouvrages très proches (même auteur, même collection, mêmes thèmes).

Cette distribution reflète la nature des vecteurs TF-IDF : des vecteurs haute dimension mais extrêmement creux, pour lesquels les recouvrements lexicaux significatifs restent limités. L'observation confirme ainsi que la similarité cosinus produit des valeurs cohérentes avec la structure linguistique du corpus.

3.3.2 Choix du seuil (threshold) pour la construction du graphe

Lors de la construction du graphe documentaire G_d , nous retenons uniquement les arêtes dont la similarité cosinus vérifie :

$$\text{similarité} \geq 0.05.$$

Ce seuil résulte d'une analyse empirique de la distribution observée :

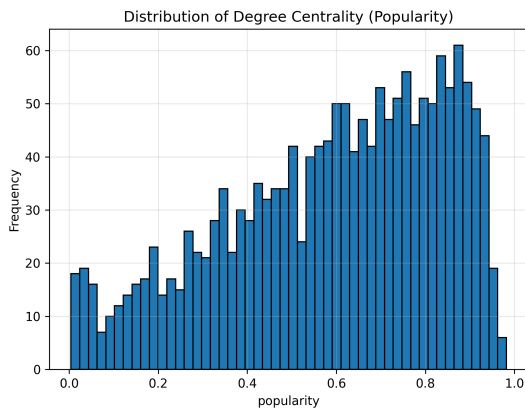
- **Si le seuil est inférieur à 0.05** : un très grand nombre d'arêtes "faiblement similaires" est ajouté, ce qui rend le graphe trop dense et introduit un bruit important dans les calculs de centralité.

- **Si le seuil dépasse 0.10** : le nombre d'arêtes chute brutalement ; de nombreuses relations sémantiques pertinentes disparaissent, ce qui fragilise la structure du graphe (risque de composantes faiblement connectées).
- **Le seuil 0.05 constitue un compromis optimal** : il permet de conserver les relations significatives tout en filtrant efficacement le bruit, assurant ainsi un graphe suffisamment structuré pour les calculs de centralité.

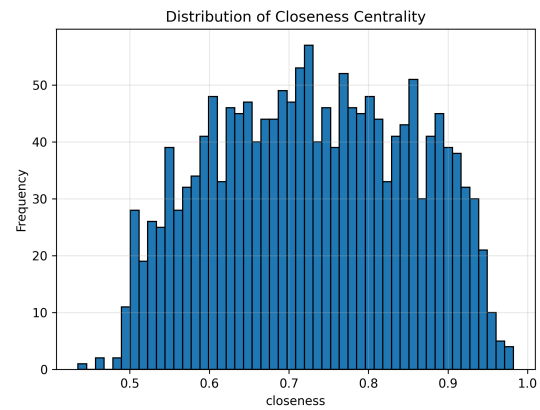
Ainsi, la valeur 0.05 représente un choix équilibré entre densité, pertinence et stabilité computationnelle.

3.4 Tests et analyse des mesures de centralité

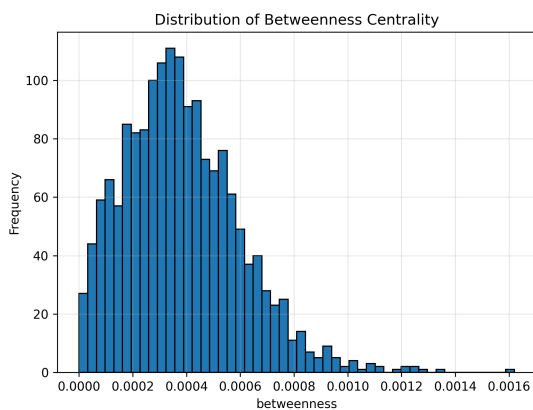
Afin de vérifier la cohérence structurelle du graphe documentaire, cette section analyse la distribution statistique des différentes mesures de centralité. L'objectif est de confirmer que les scores obtenus présentent des propriétés attendues dans les graphes issus de corpus textuels : connectivité locale, proximité globale, rôle d'intermédiation et influence structurelle.



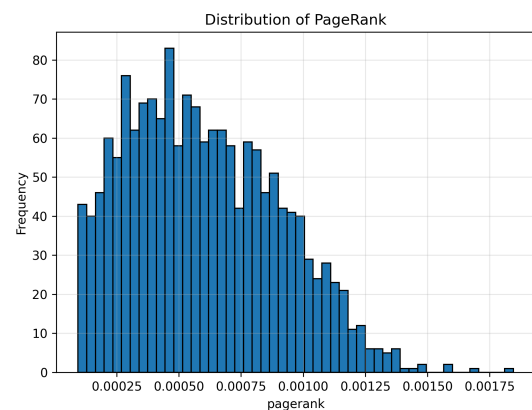
(a) Centralité de degré (Popularity)



(b) Centralité de proximité (Closeness)



(c) Centralité d'intermédiation (Betweenness)



(d) PageRank

FIGURE 5 – Distributions des quatre mesures de centralité dans le graphe documentaire

3.4.1 Distribution de la centralité de degré (Popularity)

La centralité de degré mesure combien de documents sont similaires à un document donné (similarité cosinus ≥ 0.05). L'histogramme montre que la majorité des documents possèdent

un degré moyen à élevé, ce qui indique une structure thématique en grappes. Certaines œuvres, plus transversales ou plus générales, présentent un degré nettement supérieur.

3.4.2 Distribution de la centralité de proximité (Closeness)

La centralité de proximité évalue la distance moyenne d'un document vers tous les autres. Les valeurs observées se situent majoritairement entre 0.5 et 0.9, ce qui reflète une structure de type *small-world* : la plupart des documents sont accessibles en très peu d'étapes dans le graphe, témoignant d'une bonne cohésion globale.

3.4.3 Distribution de la centralité d'intermédiarité (Betweenness)

La centralité d'intermédiarité mesure à quel point un document joue le rôle de « pont » dans les plus courts chemins entre d'autres documents. La distribution est fortement concentrée près de 0, avec seulement quelques documents jouant un rôle stratégique d'intermédiaire entre des communautés thématiques. Il s'agit d'un comportement typique dans les graphes textuels.

3.4.4 Distribution du PageRank

Le PageRank évalue l'importance globale d'un document dans le graphe. La distribution observée présente une forme en loi de puissance : beaucoup de documents possèdent un très faible PageRank, mais une minorité joue un rôle notable dans la diffusion de l'information. Cela confirme l'existence de documents « pivots » au sein du corpus.

4 Conclusion

Au terme de ce projet, nous avons non seulement construit un système complet de recherche et de recommandation en texte intégral, mais également acquis une compréhension concrète du fonctionnement interne d'un moteur de recherche. L'ensemble du processus – depuis l'acquisition et le nettoyage du corpus, en passant par la construction de l'index inversé, la vectorisation TF-IDF, le calcul de similarité cosinus, la génération du graphe documentaire et l'analyse de centralité – nous a permis d'expérimenter pour la première fois le cycle complet des algorithmes de recherche d'information sur un corpus réel. Ainsi, des notions jusque-là théoriques se sont incarnées dans des contraintes d'ingénierie observables et mesurables.

Par exemple, le modèle TF-IDF et l'espace vectoriel, pourtant simples en apparence, s'avèrent beaucoup plus complexes lors de leur mise en œuvre : effet de longue traîne dans la distribution des termes, normalisation liée à la longueur des documents, coûts de calcul et de stockage associés aux matrices creuses, compromis entre performance et précision lors du découpage TF-IDF, etc. C'est seulement face à des données réelles que l'on peut apprécier la nécessité et les limites de ces modèles classiques.

De même, les mesures de centralité, initialement apprises sous forme de formules, sont devenues de véritables outils analytiques dans le cadre du projet. Avant de construire le graphe de similarité, notre compréhension de Closeness, PageRank ou Betweenness demeurait principalement théorique. L'expérimentation sur corpus nous a montré comment ces mesures révèlent les structures thématiques, les relations entre documents et l'existence de « documents pivots ». Cette transition de la théorie à la pratique illustre pleinement la valeur des méthodes de graphes en recherche d'information.

Il convient également de rappeler que notre corpus de 1664 livres représente une échelle expérimentale modérée, loin de celle des moteurs de recherche industriels. À grande échelle, les systèmes doivent faire face à la croissance exponentielle de la taille de l'index, des dimensions vectorielles, du graphe documentaire ainsi qu'aux contraintes de mise à jour en temps réel.

Néanmoins, le traitement de ce corpus nous a permis d’entrevoir une version réduite mais représentative de l’ingénierie des moteurs de recherche.

L’évolution actuelle des systèmes de recherche tend à dépasser la simple correspondance lexicale pour s’orienter vers une compréhension plus sémantique. Contrairement au modèle TF-IDF, fondé sur la coïncidence de formes lexicales, les méthodes modernes – embeddings, modèles transformeurs légers, petits LLM – capturent des relations sémantiques plus profondes et rapprochent davantage la réponse du véritable intention de l’utilisateur. Intégrer de telles représentations sémantiques avec l’index inversé, le graphe de similarité et les mesures de centralité développés dans ce projet constituerait une voie prometteuse pour concevoir une nouvelle génération de moteurs de recherche plus intelligents et plus sensibles au contexte.

En définitive, ce projet nous a permis de construire un moteur de recherche opérationnel intégrant les composantes essentielles “recherche – classement – recommandation”. Surtout, il nous a offert l’occasion de redécouvrir, à travers des données réelles et des contraintes concrètes, les algorithmes de recherche d’information, et de valider le comportement des modèles vectoriels, des graphes documentaires et des mesures de centralité dans un cadre pratique. Ce travail constitue ainsi une base solide pour de futures explorations dans les domaines de la recherche d’information, du traitement automatique des langues et de l’analyse de graphes.